

Bloc 2 TD 1 - TDD PHP - Tests unitaires

Contexte	2
Le cours sans framework	2
Installation de Composer	7
Installation de PHPUnit et Pest	8
Les potions avec PHPUnit	12
Les potions avec Pest	17
Comparatif PHPUnit / Pest	19
Conclusion	19

Contexte

Ce TD va nous faire découvrir les **tests unitaires** en PHP avec deux frameworks de test, **PHPUnit** et **Pest**. On commence par refaire l'exemple du cours (les potions de foudre d'Harry Potter) **sans framework**, puis on installe **Composer**, **PHPUnit** et **Pest** et on refait le même exemple avec les deux frameworks.

Tout est fait avec le **PHP 8.2** de **XAMPP** et **VS Code**, le projet est disponible dans le portfolio.

Le cours sans framework

On commence par recopier le jeu de tests du cours dans un fichier **sans_framework/potions.php**. C'est un tableau de tableaux avec 3 éléments : le nom du test, les ingrédients et le nombre de potions attendu :

```
1  <?php
2
3  $tests = [
4      [
5          'nom' => '2 ingrédients, dans les quantités requises',
6          'ingredients' => ['nuages_tenebreux' => 2, 'poussiere_fee' => 1],
7          'potions' => 1
8      ],
9      [
10         'nom' => '2 ingrédients, mais pas assez du premier',
11         'ingredients' => ['nuages_tenebreux' => 1, 'poussiere_fee' => 1],
12         'potions' => 0
13     ],
14     [
15         'nom' => '2 ingrédients, dans les quantités X fois requises',
16         'ingredients' => ['nuages_tenebreux' => 8, 'poussiere_fee' => 4],
17         'potions' => 4
18     ],
19     [
20         'nom' => 'manque l\'ingrédient 1',
21         'ingredients' => ['poussiere_fee' => 1],
22         'potions' => 0
23     ],
24     [
```

Puis la première version de la fonction, qui renvoie toujours 0, et le **foreach** qui compare le nombre de potions attendu avec le résultat de la fonction. Si c'est égal on affiche **OK** en vert, sinon **KO** en rouge :

```
function nbPotionsFoudreRealisables(array $ingredients): int
{
    return 0;
}

foreach ($tests as $test) {
    if ($test['potions'] === nbPotionsFoudreRealisables($test['ingredients'])) {
        echo '<h6 style="color: green;">Test : ' . $test['nom'] . ' OK</h6>';
    } else {
        echo '<h6 style="color: red;">Test : ' . $test['nom'] . ' KO</h6>';
    }
}
```

On ouvre la page dans le navigateur

(http://localhost/public_html/TDD/TD1/sans_framework/potions.php) :

Test : 2 ingrédients, dans les quantités requises KO

Test : 2 ingrédients, mais pas assez du premier OK

Test : 2 ingrédients, dans les quantités X fois requises KO

Test : manque l'ingrédient 1 OK

Test : manque l'ingrédient 2 OK

Test : ingrédients autres OK

Test : Aucun ingrédient OK

Test : Beaucoup ingrédient 1 KO

Test : Beaucoup ingrédient 2 KO

Comme dans le cours, il y a déjà 5 tests verts juste parce que la fonction renvoie 0, les 4 autres sont rouges.

On écrit la première vraie version de la fonction : la recette est dans la fonction, on calcule pour chaque ingrédient combien de potions on peut faire et on garde le **minimum**, car c'est l'ingrédient le plus limitant qui décide :

```
51 function nbPotionsFoudreRealisables(array $ingredients): int
52 {
53     $recette = [
54         'nuages_tenebreux' => 2,
55         'poussiere_fee' => 1
56     ];
57
58     $nbPotionsParIngredient = [];
59
60     foreach ($ingredients as $ingredient => $quantite) {
61         if (isset($recette[$ingredient])) {
62             $maxPourCetIngredient = floor($quantite / $recette[$ingredient]);
63             $nbPotionsParIngredient[] = $maxPourCetIngredient;
64         }
65     }
66
67     if (count($nbPotionsParIngredient) < count($recette)) {
68         return 0;
69     }
70
71     return min($nbPotionsParIngredient);
72 }
```

Test : 2 ingrédients, dans les quantités requises OK

Test : 2 ingrédients, mais pas assez du premier OK

Test : 2 ingrédients, dans les quantités X fois requises OK

Test : manque l'ingrédient 1 OK

Test : manque l'ingrédient 2 OK

Test : ingrédients autres OK

Test : Aucun ingrédient OK

Test : Beaucoup ingrédient 1 OK

Test : Beaucoup ingrédient 2 OK

Tout est vert, la fonction est correcte et les tests “verrouillent” son bon fonctionnement.

Refactoring

Maintenant qu'on a des tests, on peut modifier le code sans avoir peur de casser quelque chose, les tests servent de filet de sécurité. Deuxième version : on parcourt la **recette** au lieu des ingrédients, comme ça si on a 1000 ingrédients qui servent à rien, on ne les parcourt pas tous :

```
function nbPotionsFoudreRealisables(array $ingredients): int
{
    $recette = [
        'nuages_tenebreux' => 2,
        'poussiere_fee' => 1
    ];

    $nbPotionsParIngredient = [];

    foreach ($recette as $ingredient => $quantiteNecessaire) {
        if (!isset($ingredients[$ingredient])) {
            return 0;
        }

        if ($ingredients[$ingredient] < $quantiteNecessaire) {
            return 0;
        }

        $nbPotionsParIngredient[] = floor($ingredients[$ingredient] / $quantiteNecessaire);
    }

    return min($nbPotionsParIngredient);
}
```

Test : 2 ingrédients, dans les quantités requises OK

Test : 2 ingrédients, mais pas assez du premier OK

Test : 2 ingrédients, dans les quantités X fois requises OK

Test : manque l'ingrédient 1 OK

Test : manque l'ingrédient 2 OK

Test : ingrédients autres OK

Test : Aucun ingrédient OK

Test : Beaucoup ingrédient 1 OK

Test : Beaucoup ingrédient 2 OK

Troisième version, avec un seul **if** grâce au **||** (OU) et le minimum qui est calculé au fur et à mesure :

```
51 function nbPotionsFoudreRealisables(array $ingredients): int
52 {
53     if (empty($ingredients)) {
54         return 0;
55     }
56
57     $recette = [
58         'nuages_tenebreux' => 2,
59         'poussiere_fee' => 1
60     ];
61
62     $nbPotions = max($ingredients);
63
64     foreach ($recette as $ingredient => $quantiteNecessaire) {
65         if (!isset($ingredients[$ingredient]) || $ingredients[$ingredient] < $quantiteNecessaire) {
66             return 0;
67         }
68
69         $nbPotions = min(
70             $nbPotions,
71             floor($ingredients[$ingredient] / $quantiteNecessaire)
72         );
73     }
74
75     return $nbPotions;
76 }
77
```

Test : 2 ingrédients, dans les quantités requises OK

Test : 2 ingrédients, mais pas assez du premier OK

Test : 2 ingrédients, dans les quantités X fois requises OK

Test : manque l'ingrédient 1 OK

Test : manque l'ingrédient 2 OK

Test : ingrédients autres OK

Test : Aucun ingrédient OK

Test : Beaucoup ingrédient 1 OK

Test : Beaucoup ingrédient 2 OK

Toujours vert, donc le comportement de la fonction n'a pas changé. Par contre on voit bien la limite : il faut recharger la page et regarder les couleurs à la main, c'est pour ça qu'on passe à un vrai framework de test.

Installation de Composer

PHPUnit et Pest s'installent avec **Composer**, le gestionnaire de dépendances de PHP. Il n'était pas installé sur mon PC, on va donc sur le site officiel (<https://getcomposer.org/download/>) et on utilise l'installation en ligne de commande, comme ça on voit bien ce qui se passe :

Download Composer Latest: v2.10.3

 PRIVATE PACKAGIST

Secure & reliable package repository

Windows Installer

The installer - which requires that you have PHP already installed - will download Composer for you and set up your PATH environment variable so you can simply call `composer` from any directory.

Download and run [Composer-Setup.exe](#) - it will install the latest composer version whenever it is executed.

Command-line installation

To quickly install Composer in the current directory, run the following script in your terminal. To automate the installation, use [the guide on installing Composer programmatically](#).

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php -r "if (hash_file('sha384', 'composer-setup.php') === 'c8b085408188070d5f52bcfe4ecfbee5f727afa458b2573b8eaa
php composer-setup.php
php -r "unlink('composer-setup.php');"
```

On se place dans le dossier PHP de XAMPP (**C:\xampp\php**) car il est déjà dans le **PATH**, comme ça la commande `composer` marchera de partout. La première commande ne passe pas :

```
PS C:\xampp\php> php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
PHP Warning: copy(https://getcomposer.org/installer): Failed to open stream: Une tentative de connexion a échoué car le parti connecté n'a pas r
épondu convenablement au-delà d'une certaine durée ou une connexion établie a échoué car l'hôte de connexion n'a pas répondu in Command line code
on line 1

Warning: copy(https://getcomposer.org/installer): Failed to open stream: Une tentative de connexion a échoué car le parti connecté n'a pas répo
ndu convenablement au-delà d'une certaine durée ou une connexion établie a échoué car l'hôte de connexion n'a pas répondu in Command line code on l
ine 1
PS C:\xampp\php>
```

PHP essaye de télécharger l'installateur en direct, mais à l'école on passe par un **proxy** et la fonction **copy()** de PHP ne l'utilise pas. On télécharge donc l'installateur avec **curl.exe** qui lui passe par le proxy :

```
PS C:\xampp\php> curl.exe -o composer-setup.php https://getcomposer.org/installer
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left   Speed
100 59524 100 59524  0    0 380.4k    0    0    0  0
PS C:\xampp\php>
```

Ensuite on vérifie l'installateur avec le hash **SHA-384** donné sur le site, puis on le lance :

```
PS C:\xampp\php> php -r "if (hash_file('sha384', 'composer-setup.php') === 'c8b085408188070d5f52bcfe4ecfbee5f727afa458b257c67c86944da1544f06fa544fd47') { echo 'Installer verified'.PHP_EOL; } else { echo 'Installer corrupt'.PHP_EOL; unlink('composer-setup.php'); }"
Installer verified
PS C:\xampp\php> php composer-setup.php
All settings correct for using Composer
Downloading...

Composer (version 2.10.3) successfully installed to: C:\xampp\php\composer.phar
Use it: php composer.phar
```

Composer 2.10.3 est installé dans *C:\xampp\php\composer.phar*. On supprime l'installateur et on crée un fichier **composer.bat** à côté pour pouvoir taper juste "composer" au lieu de "php composer.phar" :

```
PS C:\xampp\php> php -r "unlink('composer-setup.php');"
PS C:\xampp\php> Set-Content composer.bat '@php "%~dp0composer.phar" %*'
PS C:\xampp\php> composer -V
Composer version 2.10.3 2026-08-27 13:34:23
PHP version 8.2.12 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.
PS C:\xampp\php>
```

Installation de PHPUnit et Pest

Dans le dossier du TD, on crée le projet avec **composer init**, avec un autoloader **PSR-4** sur le dossier **src/** (le namespace de nos classes sera **Romain\Td1**) :

```
PS C:\xampp\htdocs\public_html\TDD\TD1> composer init --name=romain/td1 --description="TD1 tests unitaires avec PHPUnit et Pest"
-autoload=src/ -n
Writing ./composer.json
Generating autoload files
Generated autoload files
PSR-4 autoloading configured. Use "namespace Romain\Td1;" in src/
Include the Composer autoloader with: require 'vendor/autoload.php';
PS C:\xampp\htdocs\public_html\TDD\TD1>
```

On installe PHPUnit en dépendance de développement (**--dev**), car on en a besoin que pour développer, pas en production :

```
- Installing sebastian/code-unit (3.0.3): Extracting archive
- Installing sebastian/cli-parser (3.0.2): Extracting archive
- Installing phpunit/php-timer (7.0.1): Extracting archive
- Installing phpunit/php-text-template (4.0.1): Extracting archive
- Installing phpunit/php-invoker (5.0.1): Extracting archive
- Installing phpunit/php-file-iterator (5.1.1): Extracting archive
- Installing theseer/tokenizer (1.3.1): Extracting archive
- Installing nikic/php-parser (v5.9.0): Extracting archive
- Installing sebastian/lines-of-code (3.0.1): Extracting archive
- Installing sebastian/complexity (4.0.1): Extracting archive
- Installing sebastian/code-unit-reverse-lookup (4.0.1): Extracting archive
- Installing phpunit/php-code-coverage (11.0.12): Extracting archive
- Installing phar-io/version (3.2.1): Extracting archive
- Installing phar-io/manifest (2.0.4): Extracting archive
- Installing myclabs/deep-copy (1.14.0): Extracting archive
- Installing phpunit/phpunit (11.5.57): Extracting archive
5 package suggestions were added by new dependencies, use `composer suggest` to see details.
Generating autoload files
25 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^11.5 for phpunit/phpunit
PS C:\xampp\htdocs\public_html\TDD\TD1> |
```

Composer prend la version **11.5**, la version 12 demande PHP 8.3 et XAMPP est en PHP 8.2. On vérifie que PHPUnit marche :

```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\phpunit --version
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.
```

Puis **Pest**. Pest est construit par-dessus PHPUnit, Composer doit donc pouvoir changer la version de PHPUnit, d'où l'option **-W** (**--with-all-dependencies**). Pest utilise un plugin Composer, Composer nous demande si on lui fait confiance :

```
- Downloading psr/container (2.0.2)
- Downloading symfony/service-contracts (v3.7.3)
- Downloading symfony/console (v7.4.20)
- Downloading nunomaduro/termwind (v2.4.0)
- Downloading nunomaduro/collision (v8.9.5)
- Downloading symfony/process (v7.4.19)
- Downloading brianium/paratest (v7.8.6)
- Downloading pestphp/pest (v3.8.7)
pestphp/pest-plugin contains a Composer plugin which is currently not in your allow-plugins config. See https://getcomposer.org/allow-plugins
Do you trust "pestphp/pest-plugin" to execute code and wish to enable it now? (writes "allow-plugins" to composer.json) [y,n,d,?] |
```

On répond **y** et l'installation continue :

```
- Installing symfony/service-contracts (v3.7.3): Extracting archive
- Installing symfony/console (v7.4.20): Extracting archive
- Installing nunomaduro/termwind (v2.4.0): Extracting archive
- Installing nunomaduro/collision (v8.9.5): Extracting archive
- Installing symfony/process (v7.4.19): Extracting archive
- Installing brianium/paratest (v7.8.6): Extracting archive
- Installing pestphp/pest (v3.8.7): Extracting archive
5 package suggestions were added by new dependencies, use `composer suggest` to see details.
Generating autoload files
44 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^3.8 for pestphp/pest
PS C:\xampp\htdocs\public_html\TDD\TD1> |
```

Pest 3.8 est installé. On initialise Pest avec **pest --init**, ça crée le **phpunit.xml**, le fichier **tests/Pest.php**, une classe **TestCase** et deux tests d'exemple (on refuse de mettre une étoile sur GitHub) :

```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\pest --init

INFO: Preparing tests directory.

phpunit.xml ..... File created.
tests/Pest.php ..... File created.
tests/TestCase.php ..... File created.
tests/Unit/ExampleTest.php ..... File created.
tests/Feature/ExampleTest.php ..... File created.

Wanna show Pest some love by starring it on GitHub? (yes/no) [no]:
> no

Star ..... https://pithub.com/pestphp/pest
```

On lance Pest pour voir si les tests d'exemple passent, mais on a une erreur :

```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\pest

Pest\Exceptions\TestCaseClassOrTraitNotFound

The class `Tests\TestCase` was not found.

PS C:\xampp\htdocs\public_html\TDD\TD1> |
```

La classe **Tests\TestCase** n'est pas trouvée car notre **composer.json** ne sait pas où sont les classes du namespace **Tests**. On ajoute un **autoload-dev** (autoload seulement pour le développement) :

```
1  {
2      "name": "romain/td1",
3      "description": "TD1 tests unitaires avec PHPUnit et Pest",
4      "type": "project",
5      "autoload": {
6          "psr-4": {
7              "Romain\\Td1\\": "src/"
8          }
9      },
10     "autoload-dev": {
11         "psr-4": {
12             "Tests\\": "tests/"
13         }
14     },
15     "authors": [
16         {
17             "name": "RomAub",
18             "email": "aubree.rom@gmail.com"
19         }
20     ],
21     "require-dev": {
22         "phpunit/phpunit": "^11.5",
23         "pestphp/pest": "^3.8"
```

Puis **composer dump-autoload** pour régénérer l'autoload, et cette fois les 2 tests d'exemple passent :

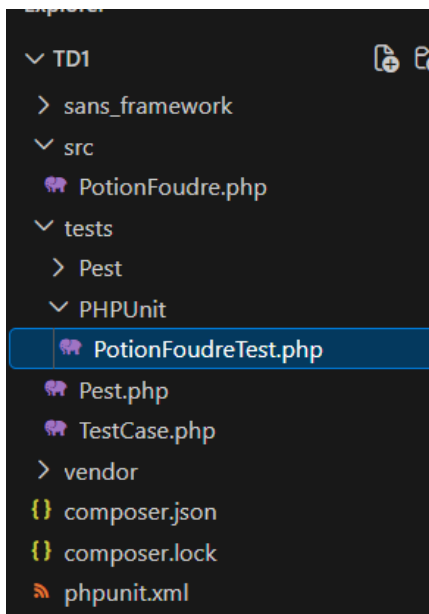
```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\pest

PASS Tests\Feature\ExampleTest
✓ example

PASS Tests\Unit\ExampleTest
✓ example

Tests: 2 passed (2 assertions)
Duration: 0.17s
```

Je supprime les tests d'exemple et j'organise le projet avec un dossier pour les tests **PHPUnit** et un dossier pour les tests **Pest** :



Les potions avec PHPUnit

On met la fonction du cours dans une classe **PotionFoudre** dans **src/**. On repart de la version qui renvoie 0, pour faire comme en **TDD** (on écrit les tests avant le code) :

```
1  <?php
2
3  namespace Romain\Td1;
4
5  class PotionFoudre
6  {
7      public function nbPotionsFoudreRealisables(array $ingredients): int
8      {
9          return 0;
10     }
11 }
12
```

Le test PHPUnit est une classe qui hérite de **TestCase**. La méthode **setUp()** est appelée avant chaque test, elle crée un nouvel objet **PotionFoudre** :

```
1  <?php
2
3  namespace Tests\PHPUnit;
4
5  use PHPUnit\Framework\Attributes\DataProvider;
6  use PHPUnit\Framework\TestCase;
7  use Romain\Td1\PotionFoudre;
8
9  final class PotionFoudreTest extends TestCase
10 {
11     private PotionFoudre $potion;
12
13     protected function setUp(): void
14     {
15         parent::setUp();
16         $this->potion = new PotionFoudre();
17     }
18 }
```

Au lieu d'écrire 9 méthodes de test, on utilise un **DataProvider** : une méthode statique qui renvoie le jeu de tests du cours, et PHPUnit lance la méthode de test une fois pour chaque ligne. On utilise **assertSame** qui vérifie la valeur et le type, comme le **===** du cours :

```
19     public static function jeuDeTests(): array
20     {
21         return [
22             '2 ingrédients, dans les quantités requises' => [['nuages_tenebreux' => 2, 'poussiere_fee' => 1], 1],
23             '2 ingrédients, mais pas assez du premier' => [['nuages_tenebreux' => 1, 'poussiere_fee' => 1], 0],
24             '2 ingrédients, dans les quantités X fois requises' => [['nuages_tenebreux' => 8, 'poussiere_fee' => 4], 4],
25             'manque 1\ingrédient 1' => [['poussiere_fee' => 1], 0],
26             'manque 1\ingrédient 2' => [['nuages_tenebreux' => 2], 0],
27             'ingrédients autres' => [['queue_lezard' => 5, 'eau_jouvence' => 2], 0],
28             'Aucun ingrédient' => [], 0],
29             'Beaucoup ingrédient 1' => [['nuages_tenebreux' => 250, 'poussiere_fee' => 1], 1],
30             'Beaucoup ingrédient 2' => [['nuages_tenebreux' => 2, 'poussiere_fee' => 120], 1],
31         ];
32     }
33
34     #[DataProvider('jeuDeTests')]
35     public function testNbPotionsFoudreRealisables(array $ingredients, int $potionsAttendues): void
36     {
37         $this->assertSame($potionsAttendues, $this->potion->nbPotionsFoudreRealisables($ingredients));
38     }
39 }
```

Dans le **phpunit.xml** on déclare 2 suites de tests, une pour PHPUnit et une pour Pest, pour pouvoir les lancer séparément :

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
4     bootstrap="vendor/autoload.php"
5     colors="true"
6 >
7     <testsuites>
8         <testsuite name="PHPUnit">
9             <directory suffix="Test.php">./tests/PHPUnit</directory>
10        </testsuite>
11        <testsuite name="Pest">
12            <directory suffix="Test.php">./tests/Pest</directory>
13        </testsuite>
14    </testsuites>
15    <source>
16        <include>
17            <directory>src</directory>
18        </include>
19    </source>
```

On lance les tests PHPUnit avec **.\vendor\bin\phpunit --testsuite PHPUnit** :

```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.56 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD1\phpunit.xml

.F.F....FF                                         9 / 9 (100%)

Time: 00:00.026, Memory: 8.00 MB

There were 4 failures:

1) Tests\PHPUnit\PotionFoudreTest::testNbPotionsFoudreRealisables with data set "2 ingrédients, dans les quantités requises"
Failed asserting that 0 is identical to 1.

C:\xampp\htdocs\public_html\TDD\TD1\tests\PHPUnit\PotionFoudreTest.php:37

2) Tests\PHPUnit\PotionFoudreTest::testNbPotionsFoudreRealisables with data set "2 ingrédients, dans les quantités x fois req
Failed asserting that 0 is identical to 4.

C:\xampp\htdocs\public_html\TDD\TD1\tests\PHPUnit\PotionFoudreTest.php:37

3) Tests\PHPUnit\PotionFoudreTest::testNbPotionsFoudreRealisables with data set "Beaucoup ingrédient 1" ([250, 1], 1)
Failed asserting that 0 is identical to 1.

C:\xampp\htdocs\public_html\TDD\TD1\tests\PHPUnit\PotionFoudreTest.php:37

4) Tests\PHPUnit\PotionFoudreTest::testNbPotionsFoudreRealisables with data set "Beaucoup ingrédient 2" ([2, 120], 1)
Failed asserting that 0 is identical to 1.

C:\xampp\htdocs\public_html\TDD\TD1\tests\PHPUnit\PotionFoudreTest.php:37

FAILURES!
Tests: 9, Assertions: 9, Failures: 4.
```

On retrouve exactement les 4 tests KO du cours. Les **.** sont les tests qui passent et les **F** les échecs, et pour chaque échec PHPUnit affiche la valeur attendue et celle obtenue.

On met la première version de la fonction du cours :

```
5 class PotionFoudre
6 {
7     public function nbPotionsFoudreRealisables(array $ingredients): int
8     {
9         $recette = [
10             'nuages_tenebreux' => 2,
11             'poussiere_fee' => 1
12         ];
13
14         $nbPotionsParIngredient = [];
15
16         foreach ($ingredients as $ingredient => $quantite) {
17             if (isset($recette[$ingredient])) {
18                 $maxPourCetIngredient = floor($quantite / $recette[$ingredient]);
19                 $nbPotionsParIngredient[] = $maxPourCetIngredient;
20             }
21         }
22
23         if (count($nbPotionsParIngredient) < count($recette)) {
24             return 0;
25         }
26
27         return min($nbPotionsParIngredient);
28     }
29 }
```

```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.56 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD1\phpunit.xml

.....                                     9 / 9 (100%)

Time: 00:00.016, Memory: 8.00 MB

OK (9 tests, 9 assertions)
PS C:\xampp\htdocs\public_html\TDD\TD1>
```

9 tests, 9 assertions, tout est vert.

Refactoring

Comme dans le cours, on refactorise. En plus de la 3ème version du cours, la recette devient une **constante de classe** et j'utilise **intdiv()** (division entière qui renvoie un int) au lieu de **floor()** qui renvoie un float :

```
5 class PotionFoudre
6 {
7     private const RECETTE = [
8         'nuages_tenebreux' => 2,
9         'poussiere_fee' => 1,
10    ];
11
12    public function nbPotionsFoudreRealisables(array $ingredients): int
13    {
14        if (empty($ingredients)) {
15            return 0;
16        }
17
18        $nbPotions = max($ingredients);
19
20        foreach (self::RECETTE as $ingredient => $quantiteNecessaire) {
21            if (!isset($ingredients[$ingredient]) || $ingredients[$ingredient] < $quantiteNecessaire) {
22                return 0;
23            }
24
25            $nbPotions = min($nbPotions, intdiv($ingredients[$ingredient], $quantiteNecessaire));
26        }
27
28        return $nbPotions;
29    }
30 }
31
```

Avec l'option **--testdox**, PHPUnit affiche le nom de chaque test (ici le nom de chaque ligne du DataProvider), c'est plus lisible :

```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\phpunit --testsuite PHPUnit --testdox
PHPUnit 11.5.56 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD1\phpunit.xml

.....
9 / 9 (100%)

Time: 00:00.014, Memory: 8.00 MB

Potion Foudre (Tests\PHPUnit\PotionFoudre)
✓ Nb potions foudre realisables with 2 ingrédients, dans les quantités requises
✓ Nb potions foudre realisables with 2 ingrédients, mais pas assez du premier
✓ Nb potions foudre realisables with 2 ingrédients, dans les quantités X fois requises
✓ Nb potions foudre realisables with manque l'ingrédient 1
✓ Nb potions foudre realisables with manque l'ingrédient 2
✓ Nb potions foudre realisables with ingrédients autres
✓ Nb potions foudre realisables with Aucun ingrédient
✓ Nb potions foudre realisables with Beaucoup ingrédient 1
✓ Nb potions foudre realisables with Beaucoup ingrédient 2

OK (9 tests, 9 assertions)
```

Les potions avec Pest

Avec Pest, pas besoin de classe : on écrit des fonctions **it()** ou **test()** et on vérifie avec **expect(...)->toBe(...)**. Le jeu de tests se met avec **->with()**, c'est l'équivalent du **DataProvider**. **beforeEach()** remplace le **setUp()**. J'ai rajouté un test en plus :

```
1  <?php
2
3  use Romain\Td1\PotionFoudre;
4
5  beforeEach(function () {
6      $this->potion = new PotionFoudre();
7  });
8
9  it('calcule le nombre de potions de foudre réalisables', function (array $ingredients, int $potionsAttendues) {
10     expect($this->potion->nbPotionsFoudreRealisables($ingredients))->toBe($potionsAttendues);
11 })->with([
12     '2 ingrédients, dans les quantités requises' => [['nuages_tenebreux' => 2, 'poussiere_fee' => 1], 1],
13     '2 ingrédients, mais pas assez du premier' => [['nuages_tenebreux' => 1, 'poussiere_fee' => 1], 0],
14     '2 ingrédients, dans les quantités X fois requises' => [['nuages_tenebreux' => 8, 'poussiere_fee' => 4], 4],
15     'manque 1\'ingrédient 1' => [['poussiere_fee' => 1], 0],
16     'manque 1\'ingrédient 2' => [['nuages_tenebreux' => 2], 0],
17     'ingrédients autres' => [['queue_lezard' => 5, 'eau_jouvence' => 2], 0],
18     'Aucun ingrédient' => [], 0],
19     'Beaucoup ingrédient 1' => [['nuages_tenebreux' => 250, 'poussiere_fee' => 1], 1],
20     'Beaucoup ingrédient 2' => [['nuages_tenebreux' => 2, 'poussiere_fee' => 120], 1],
21 ]);
22
23 test('une potion ne peut pas être réalisée sans recette complète', function () {
24     expect($this->potion->nbPotionsFoudreRealisables(['nuages_tenebreux' => 10]))->toBe(0)
25     ->and($this->potion->nbPotionsFoudreRealisables(['poussiere_fee' => 10]))->toBe(0);
26 });
```

On lance la suite Pest :

```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\pest --testsuite Pest

PASS Tests\Pest\PotionFoudreTest
✓ it calcule le nombre de potions de foudre réalisables with dataset "2 ingrédients, dans les quantités requises"
✓ it calcule le nombre de potions de foudre réalisables with dataset "2 ingrédients, mais pas assez du premier"
✓ it calcule le nombre de potions de foudre réalisables with dataset "2 ingrédients, dans les quantités X fois requises"
✓ it calcule le nombre de potions de foudre réalisables with dataset "manque l'ingrédient 1"
✓ it calcule le nombre de potions de foudre réalisables with dataset "manque l'ingrédient 2"
✓ it calcule le nombre de potions de foudre réalisables with dataset "ingrédients autres"
✓ it calcule le nombre de potions de foudre réalisables with dataset "Aucun ingrédient"
✓ it calcule le nombre de potions de foudre réalisables with dataset "Beaucoup ingrédient 1"
✓ it calcule le nombre de potions de foudre réalisables with dataset "Beaucoup ingrédient 2"
✓ une potion ne peut pas être réalisée sans recette complète

Tests: 10 passed (11 assertions)
Duration: 0.22s
```

Pour voir à quoi ressemble une erreur avec Pest, je change volontairement le résultat attendu du dernier test (2 au lieu de 1) :

```
✓ it calcule le nombre de potions de foudre réalisables with dataset "Beaucoup ingrédient 1"
✗ it calcule le nombre de potions de foudre réalisables with dataset "Beaucoup ingrédient 2"
✓ une potion ne peut pas être réalisée sans recette complète

FAILED Tests\Pest\PotionFoudreTest > it calcule le nombre de potions de foudre réalisables with dataset "Beaucoup ingrédient 2"
Failed asserting that 1 is identical to 2.

at tests\Pest\PotionFoudreTest.php:10
 6 |     $this->potion = new PotionFoudre();
 7 | });
 8 |
 9 | it('calcule le nombre de potions de foudre réalisables', function (array $ingrédients, int $potionsAttendues) {
→ 10 |     expect($this->potion->nbPotionsFoudreRealisables($ingrédients))->toBe($potionsAttendues);
11 | })->with([
12 |     '2 ingrédients, dans les quantités requises' => [['nuages_tenebreux' => 2, 'poussiere_fee' => 1], 1],
13 |     '2 ingrédients, mais pas assez du premier' => [['nuages_tenebreux' => 1, 'poussiere_fee' => 1], 0],
14 |     '2 ingrédients, dans les quantités X fois requises' => [['nuages_tenebreux' => 8, 'poussiere_fee' => 4], 4],
15 | ]);

1 tests\Pest\PotionFoudreTest.php:10

Tests: 1 failed, 9 passed (11 assertions)
Duration: 0.19s
```

Pest montre directement la ligne du test qui échoue avec le code autour, c'est plus clair que PHPUnit. Je remets la bonne valeur.

Pest sait aussi lancer les tests écrits pour PHPUnit, avec juste `.\vendor\bin\pest` on lance les 2 suites :

```
PS C:\xampp\htdocs\public_html\TDD\TD1> .\vendor\bin\pest

PASS Tests\PHPUnit\PotionFoudreTest
✓ nb potions foudre réalisables with data set "2 ingrédients, dans les quantités requises"
✓ nb potions foudre réalisables with data set "2 ingrédients, mais pas assez du premier"
✓ nb potions foudre réalisables with data set "2 ingrédients, dans les quantités x fois requises"
✓ nb potions foudre réalisables with data set "manque l'ingrédient 1"
✓ nb potions foudre réalisables with data set "manque l'ingrédient 2"
✓ nb potions foudre réalisables with data set "ingrédients autres"
✓ nb potions foudre réalisables with data set "aucun ingrédient"
✓ nb potions foudre réalisables with data set "beaucoup ingrédient 1"
✓ nb potions foudre réalisables with data set "beaucoup ingrédient 2"

PASS Tests\Pest\PotionFoudreTest
✓ it calcule le nombre de potions de foudre réalisables with dataset "2 ingrédients, dans les quantités requises"
✓ it calcule le nombre de potions de foudre réalisables with dataset "2 ingrédients, mais pas assez du premier"
✓ it calcule le nombre de potions de foudre réalisables with dataset "2 ingrédients, dans les quantités x fois requises"
✓ it calcule le nombre de potions de foudre réalisables with dataset "manque l'ingrédient 1"
✓ it calcule le nombre de potions de foudre réalisables with dataset "manque l'ingrédient 2"
✓ it calcule le nombre de potions de foudre réalisables with dataset "ingrédients autres"
✓ it calcule le nombre de potions de foudre réalisables with dataset "Aucun ingrédient"
✓ it calcule le nombre de potions de foudre réalisables with dataset "Beaucoup ingrédient 1"
✓ it calcule le nombre de potions de foudre réalisables with dataset "Beaucoup ingrédient 2"
✓ une potion ne peut pas être réalisée sans recette complète

Tests: 19 passed (20 assertions)
Duration: 0.20s
```

19 tests, tous verts.

Comparatif PHPUnit / Pest

	PHPUnit	Pest
Écriture des tests	Une classe qui hérite de TestCase, une méthode par test	Des fonctions it() / test(), pas de classe
Vérifications	\$this->assertSame(), assertEquals()...	expect(...)->toBe(), toBeTrue()...
Jeu de données	#[DataProvider] + méthode statique	->with([...])
Affichage	Classique (. F E), --testdox pour les noms	Plus lisible, montre le code du test qui échoue
Installation	composer require --dev phpunit/phpunit	composer require --dev pestphp/pest + pest --init

Pest utilise PHPUnit en dessous, donc les deux sont compatibles : un projet peut avoir des tests PHPUnit et des tests Pest, et Pest lance les deux.

Conclusion

Ce TD nous a fait installer **Composer**, **PHPUnit** et **Pest** et refaire l'exemple des potions du cours avec les deux frameworks. On a vu que les tests permettent de modifier le code sans risque. Les deux frameworks font la même chose, PHPUnit est plus classique avec ses classes, Pest est plus court à écrire et son affichage est plus agréable.